

practical4

BankAccount.java

```
/**
 * Account class representing a simple bank account with account number,
 * holder name and balance. Demonstrates default and parameterized constructors.
 */
class Account {
    // Account number that uniquely identifies this bank account.
    long accountNumber;
    // Name of the person who owns this bank account.
    String accountHolderName;
    // Current amount of money available in the account.
    double balance;

    /**
     * Default constructor: creates an empty Account using placeholder values.
     */
    Account() {
        // Store 0 as a placeholder account number because none was supplied.
        accountNumber = 0;
        // Store "Unknown" as the holder name because no name was supplied.
        accountHolderName = "Unknown";
        // Start the balance at 0.0 because no opening deposit was made.
        balance = 0.0;
    }

    /**
     * Parameterized constructor: creates an Account using the values passed in.
     */
    Account(long accNo, String name, double openingBalance) {
        // Copy the supplied account number into the instance field.
        accountNumber = accNo;
        // Copy the supplied holder name into the instance field.
        accountHolderName = name;
        // Store the opening balance as the starting balance of the account.
        balance = openingBalance;
    }

    /**
     * Adds the specified amount to the current balance.
     */
    void deposit(double amount) {
        // Add the deposited amount to the balance stored in the object.
        balance += amount;
        // Print a confirmation message showing how much was deposited.
        System.out.println("Deposited: " + amount);
    }

    /**
     * Withdraws the amount but only if sufficient balance is available.
     */
    void withdraw(double amount) {
        // Check whether the account has enough money before allowing the withdrawal.
        if (amount <= balance) {
            // Deduct the withdrawn amount from the available balance.
            balance -= amount;
            // Print a confirmation message showing how much was withdrawn.
            System.out.println("Withdrawn: " + amount);
        }
    }
}
```

```

        } else {
            // The account is short on funds, so tell the user the withdrawal failed.
            System.out.println("Insufficient balance. Withdrawal failed.");
        }
    }

    /**
     * Displays the current balance of the account.
     */
    void displayBalance() {
        // Print the value currently stored in the balance field.
        System.out.println("Current Balance: " + balance);
    }

    /**
     * Displays the full details of the account (number, holder name and balance).
     */
    void displayAccountDetails() {
        // Print the account number stored in the object.
        System.out.println("Account Number: " + accountNumber);
        // Print the name of the account holder.
        System.out.println("Account Holder: " + accountHolderName);
        // Reuse displayBalance() to show the current balance.
        displayBalance();
    }
}

/**
 * Demonstrates the BankAccount management system using classes and objects.
 */
public class BankAccount {

    /**
     * Main method: creates two Account objects (one via each constructor),
     * performs deposits and withdrawals, and displays their details.
     */
    public static void main(String[] args) {
        // Create the first account using the default constructor (all placeholder values).
        Account a1 = new Account();
        // Print a header showing that this section belongs to the default account.
        System.out.println("--- Account 1 (Default Constructor) ---");
        // Show the initial details of the default account.
        a1.displayAccountDetails();
        // Deposit 5000 rupees into the first account.
        a1.deposit(5000);
        // Withdraw 2000 rupees from the first account.
        a1.withdraw(2000);
        // Show the details again to see the effect of the deposit and withdrawal.
        a1.displayAccountDetails();

        // Print a blank line to separate the two account sections.
        System.out.println();

        // Create the second account using the parameterized constructor.
        Account a2 = new Account(1023456789L, "Harkeerat", 10000);
        // Print a header showing that this section belongs to the parameterized account.
        System.out.println("--- Account 2 (Parameterized Constructor) ---");
        // Show the initial details of the second account.
        a2.displayAccountDetails();
        // Try to withdraw 12000 with only 10000 available (this should fail).
        a2.withdraw(12000);
        // Withdraw 4000 rupees (this should succeed).
        a2.withdraw(4000);
    }
}

```

```
        // Show only the current balance of the second account.  
        a2.displayBalance();  
        // Show the complete final details of the second account.  
        a2.displayAccountDetails();  
    }  
}
```

BankAccountArray.java

```
// Import Scanner: used to read account details (account number, name, balance) typed on the keyboard.
import java.util.Scanner;

/**
 * Account class representing a simple bank account with account number,
 * holder name and balance.
 */
class BankAccount {
    // Account number that uniquely identifies this bank account.
    long accountNumber;
    // Name of the person who owns this bank account.
    String holderName;
    // Current amount of money available in the account.
    double balance;

    /**
     * Parameterized constructor: creates a BankAccount using the values passed in.
     */
    BankAccount(long accNo, String name, double openingBalance) {
        // Copy the supplied account number into the instance field.
        accountNumber = accNo;
        // Copy the supplied holder name into the instance field.
        holderName = name;
        // Store the opening balance as the starting balance of the account.
        balance = openingBalance;
    }

    /**
     * Adds the specified amount to the current balance.
     */
    void deposit(double amount) {
        // Add the deposited amount to the balance stored in the object.
        balance += amount;
        // Print a confirmation message showing how much was deposited.
        System.out.println("Deposited: " + amount);
    }

    /**
     * Withdraws the amount but only if sufficient balance is available.
     */
    void withdraw(double amount) {
        // Check whether the account has enough money before allowing the withdrawal.
        if (amount <= balance) {
            // Deduct the withdrawn amount from the available balance.
            balance -= amount;
            // Print a confirmation message showing how much was withdrawn.
            System.out.println("Withdrawn: " + amount);
        } else {
            // The account is short on funds, so tell the user the withdrawal failed.
            System.out.println("Insufficient balance. Withdrawal failed.");
        }
    }

    /**
     * Displays the full details of the account (number, holder name and balance).
     */
    void displayDetails() {
        // Print the account number stored in the object.
        System.out.println("Account Number: " + accountNumber);
    }
}
```

```

        // Print the name of the account holder.
        System.out.println("Account Holder: " + holderName);
        // Print the current balance of the account.
        System.out.println("Current Balance: " + balance);
    }
}

/**
 * Demonstrates an array of BankAccount objects for 5 customers.
 */
public class BankAccountArray {

    /**
     * Main method: reads details of 5 customers from the keyboard into an array,
     * then performs a deposit and a withdrawal on each account and shows the results.
     */
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // Create an array that can hold 5 BankAccount objects (initially all null).
        BankAccount[] customers = new BankAccount[5];

        // Accept details for 5 customers
        // Loop through each position of the array and fill it with a customer's details.
        for (int i = 0; i < customers.length; i++) {
            // Print a heading with the current customer number (i+1 because the loop starts at 0).
            System.out.println("\n--- Enter details for Customer " + (i + 1) + " ---");
            // Ask the user to enter the account number.
            System.out.print("Enter Account Number: ");
            // Read the long value (the account number) typed by the user.
            long accNo = sc.nextLong();
            // Consume the leftover newline character so the next read works correctly.
            sc.nextLine();
            // Ask the user to enter the account holder name.
            System.out.print("Enter Account Holder Name: ");
            // Read the whole line (the holder name) typed by the user.
            String name = sc.nextLine();
            // Ask the user to enter the opening balance.
            System.out.print("Enter Opening Balance: ");
            // Read the double value (the opening balance) typed by the user.
            double bal = sc.nextDouble();
            // Create a new BankAccount from the entered values and store it at position i.
            customers[i] = new BankAccount(accNo, name, bal);
        }

        // Perform deposit and withdrawal for each customer
        // Loop again over every account in the array.
        for (int i = 0; i < customers.length; i++) {
            // Print a heading showing the current customer number.
            System.out.println("\n--- Customer " + (i + 1) + " ---");
            // Deposit 1000 rupees into the current customer's account.
            customers[i].deposit(1000);
            // Withdraw 500 rupees from the current customer's account.
            customers[i].withdraw(500);
            // Display the final details of the current customer's account.
            customers[i].displayDetails();
        }

        // Close the Scanner to release the keyboard resource.
        sc.close();
    }
}

```

OnlineShopping.java

```
// Import Scanner: used to read input typed on the keyboard (available for future interactive features).
import java.util.Scanner;

/**
 * Base class User storing basic user information.
 */
class User {
    // Unique identifier assigned to the user.
    int userID;
    // Name of the user.
    String userName;
    // Mobile phone number of the user.
    long mobileNumber;

    /**
     * Parameterized constructor: creates a User using the values passed in.
     */
    User(int id, String name, long mobile) {
        // Copy the supplied ID into the instance field.
        userID = id;
        // Copy the supplied name into the instance field.
        userName = name;
        // Copy the supplied mobile number into the instance field.
        mobileNumber = mobile;
    }

    /**
     * Displays the basic details of the user.
     */
    void displayUserDetails() {
        // Print the user ID stored in the object.
        System.out.println("User ID: " + userID);
        // Print the user name stored in the object.
        System.out.println("User Name: " + userName);
        // Print the mobile number stored in the object.
        System.out.println("Mobile Number: " + mobileNumber);
    }
}

/**
 * Intermediate class Customer inheriting from User.
 * Adds address and order ID.
 */
class Customer extends User {
    // Delivery address of the customer.
    String customerAddress;
    // Order ID associated with the customer's purchase.
    int orderID;

    /**
     * Parameterized constructor: forwards the basic user data to the User class
     * via super and stores the address and order ID locally.
     */
    Customer(int id, String name, long mobile, String address, int order) {
        // Call the parent class constructor to set ID, name and mobile number.
        super(id, name, mobile);
        // Store the delivery address in this object's field.
        customerAddress = address;
        // Store the order ID in this object's field.
    }
}
```

```

        orderID = order;
    }

    /**
     * Displays the customer details including the inherited user details.
     */
    void displayCustomerDetails() {
        // Display the basic user details using the inherited method.
        displayUserDetails();
        // Print the customer's delivery address.
        System.out.println("Customer Address: " + customerAddress);
        // Print the customer's order ID.
        System.out.println("Order ID: " + orderID);
    }
}

/**
 * Most derived class PremiumCustomer inheriting from Customer.
 * Adds membership type and discount percentage.
 */
class PremiumCustomer extends Customer {
    // Type of membership held by this premium customer (e.g., "Gold").
    String membershipType;
    // Percentage of discount offered to this premium customer.
    double discountPercentage;

    /**
     * Parameterized constructor: forwards the customer data to the parent class
     * via super and stores the membership type and discount locally.
     */
    PremiumCustomer(int id, String name, long mobile, String address, int order,
                     String membership, double discount) {
        // Call the parent (Customer) constructor with the first five values.
        super(id, name, mobile, address, order);
        // Store the membership type in this object's field.
        membershipType = membership;
        // Store the discount percentage in this object's field.
        discountPercentage = discount;
    }

    /**
     * Calculates the discount amount on the given order amount.
     */
    double calculateDiscount(double orderAmount) {
        // Multiply the order amount by the discount percentage and divide by 100.
        return orderAmount * discountPercentage / 100;
    }

    /**
     * Displays the final bill amount after applying the discount.
     */
    void displayFinalBill(double orderAmount) {
        // Compute the discount using the calculateDiscount() method.
        double discount = calculateDiscount(orderAmount);
        // Print the original order amount.
        System.out.println("Order Amount: " + orderAmount);
        // Print the discount percentage and the computed discount amount.
        System.out.println("Discount (" + discountPercentage + "%): " + discount);
        // Print the final bill after subtracting the discount from the order amount.
        System.out.println("Final Bill Amount: " + (orderAmount - discount));
    }
}

/**

```

```

    * Displays the complete details of the premium customer.
    */
void displayPremiumDetails() {
    // Display the customer details using the inherited method.
    displayCustomerDetails();
    // Print the membership type of the premium customer.
    System.out.println("Membership Type: " + membershipType);
    // Print the discount percentage offered to the premium customer.
    System.out.println("Discount Percentage: " + discountPercentage + "%");
}
}

/**
 * Demonstrates multilevel inheritance: User -> Customer -> PremiumCustomer.
 */
public class OnlineShopping {

    /**
     * Main method: creates a PremiumCustomer object and demonstrates the full
     * inheritance chain by displaying details and calculating the final bill.
     */
    public static void main(String[] args) {
        // Create a premium customer with all details used across the inheritance chain.
        PremiumCustomer pc = new PremiumCustomer(201, "Harkeerat", 9876543210L,
            "Mithibai College, Mumbai", 5001, "Gold", 15);

        // Print a header showing the premium customer's full details.
        System.out.println("--- Premium Customer Details ---");
        // Display the complete details of the premium customer.
        pc.displayPremiumDetails();

        // Print a blank line to separate the two output sections.
        System.out.println();
        // Print a header showing the bill calculation section.
        System.out.println("--- Bill Calculation ---");
        // Display the final bill for an order amount of 5000.
        pc.displayFinalBill(5000);
    }
}

```

PaymentGateway.java

```
/**
 * Abstract class Payment representing a digital payment transaction.
 */
abstract class Payment {
    // The amount of money being paid in this transaction.
    double amount;

    /**
     * Parameterized constructor: stores the transaction amount.
     */
    Payment(double amount) {
        // Copy the passed-in amount into the instance field of this object.
        this.amount = amount;
    }

    /**
     * Concrete method that prints the basic transaction total.
     */
    void displayReceipt() {
        // Print the amount currently stored in the field.
        System.out.println("Transaction Amount: " + amount);
    }

    /**
     * Abstract method - each payment type processes funds differently,
     * so every subclass must provide its own implementation.
     */
    abstract void processPayment();
}

/**
 * Subclass CryptoPayment that extends Payment for blockchain transfers.
 */
class CryptoPayment extends Payment {
    // The blockchain address that will receive the payment.
    String walletAddress;

    /**
     * Parameterized constructor: forwards the amount to the Payment class
     * via super and stores the wallet address locally.
     */
    CryptoPayment(double amount, String walletAddress) {
        // Call the parent class constructor to store the payment amount.
        super(amount);
        // Store the wallet address in this object's field.
        this.walletAddress = walletAddress;
    }

    /**
     * Overrides the abstract processPayment() method to implement a
     * blockchain-based transfer for cryptocurrency payments.
     */
    @Override
    void processPayment() {
        // Print a confirmation showing the amount and the target wallet address.
        System.out.println("Secure blockchain transfer of " + amount +
            " confirmed to wallet: " + walletAddress);
    }
}
```

```
/**
 * Demonstrates the abstract class concept using a Digital Payment Gateway.
 */
public class PaymentGateway {

    /**
     * Main method: creates a CryptoPayment object and demonstrates
     * both the concrete receipt method and the overridden payment method.
     */
    public static void main(String[] args) {
        // Create a crypto payment of 250.75 units to the given blockchain wallet address.
        CryptoPayment payment = new CryptoPayment(250.75, "0x3f9a1cB7e2D8a1f6b4C9E7d2A5B8e4F1c0D3a7b2");

        // Call the concrete method (inherited from Payment) to show the receipt.
        payment.displayReceipt();
        // Call the overridden method to process the blockchain transfer.
        payment.processPayment();
    }
}
```

SmartHome.java

```
// Import Scanner: used to read input typed on the keyboard (available for future interactive features).
import java.util.Scanner;

/**
 * Base class Device with device ID, name and power status.
 */
class Device {
    // Unique identification number assigned to the device.
    int deviceID;
    // Human-readable name of the device (e.g., "Living Room Light").
    String deviceName;
    // Stores whether the device is currently switched on (true = ON, false = OFF).
    boolean powerStatus; // true = ON, false = OFF

    /**
     * Parameterized constructor: initializes the device ID and name,
     * and starts the device in the OFF state.
     */
    Device(int id, String name) {
        // Store the passed-in id into the instance field.
        deviceID = id;
        // Store the passed-in name into the instance field.
        deviceName = name;
        // A brand-new device always starts switched off.
        powerStatus = false;
    }

    /**
     * Switches the device on by setting the power status to true.
     */
    void turnOn() {
        // Update the power status to ON.
        powerStatus = true;
        // Print a message confirming that the device is now ON.
        System.out.println(deviceName + " is turned ON.");
    }

    /**
     * Switches the device off by setting the power status to false.
     */
    void turnOff() {
        // Update the power status to OFF.
        powerStatus = false;
        // Print a message confirming that the device is now OFF.
        System.out.println(deviceName + " is turned OFF.");
    }

    /**
     * Displays the ID, name and current power status of the device.
     */
    void displayInfo() {
        // Print the device ID stored in the object.
        System.out.println("Device ID: " + deviceID);
        // Print the device name stored in the object.
        System.out.println("Device Name: " + deviceName);
        // Use a ternary to print ON when powerStatus is true, otherwise OFF.
        System.out.println("Power Status: " + (powerStatus ? "ON" : "OFF"));
    }
}
```

```

/**
 * Derived class SmartLight inheriting from Device.
 * Adds brightness level and color mode.
 */
class SmartLight extends Device {
    // Stores the brightness level of the light as a percentage (0-100).
    int brightnessLevel;
    // Stores the colour mode of the light (e.g., "Warm White").
    String colorMode;

    /**
     * Parameterized constructor: forwards the ID and name to the Device class
     * via super and stores the brightness and colour mode locally.
     */
    SmartLight(int id, String name, int brightness, String color) {
        // Call the parent class constructor to set the device ID and name.
        super(id, name);
        // Store the brightness level in this object's field.
        brightnessLevel = brightness;
        // Store the colour mode in this object's field.
        colorMode = color;
    }

    /**
     * Changes the brightness of the light to the given level.
     */
    void changeBrightness(int level) {
        // Update the brightness field with the new level.
        brightnessLevel = level;
        // Print a confirmation showing the new brightness percentage.
        System.out.println("Brightness set to " + brightnessLevel + "%");
    }

    /**
     * Overrides the parent displayInfo() to also show brightness and colour mode.
     */
    @Override
    void displayInfo() {
        // Reuse the parent class method to print the basic device details.
        super.displayInfo();
        // Print the current brightness level.
        System.out.println("Brightness Level: " + brightnessLevel + "%");
        // Print the current colour mode.
        System.out.println("Color Mode: " + colorMode);
    }
}

/**
 * Demonstrates single inheritance using a Smart Home Automation System.
 */
public class SmartHome {

    /**
     * Main method: creates a SmartLight, turns it on, changes its brightness
     * and displays its status at each stage.
     */
    public static void main(String[] args) {
        // Create a SmartLight with an ID, name, starting brightness and colour mode.
        SmartLight light = new SmartLight(101, "Living Room Light", 50, "Warm White");

        // Print a header showing the current status section.
        System.out.println("--- SmartLight Status ---");
    }
}

```

```
// Display the initial details before the light is switched on.
light.displayInfo();

// Print a blank line to separate the output sections.
System.out.println();
// Switch the light on using the inherited turnOn() method.
light.turnOn();
// Change the brightness to 80 percent.
light.changeBrightness(80);
// Display the updated details after switching on and changing brightness.
light.displayInfo();

// Print a blank line to separate the output sections.
System.out.println();
// Switch the light off using the inherited turnOff() method.
light.turnOff();
}
}
```

StaticKeyword.java

```
/**
 * Student class demonstrating the use of the static keyword.
 */
class Student {
    // Roll number that uniquely identifies each student (stored per object).
    int rollNumber;
    // Name of the student (stored per object).
    String studentName;

    // Static data members shared by all objects (one single copy for the whole class).
    static String collegeName = "Mithibai College";
    static int count = 0;

    /**
     * Parameterized constructor: creates a Student with the given roll number and name.
     */
    Student(int roll, String name) {
        // Copy the passed-in roll number into the instance field.
        rollNumber = roll;
        // Copy the passed-in name into the instance field.
        studentName = name;
        // Increment the shared static counter for every new Student object created.
        count++;
    }

    /**
     * Displays the details of this particular student.
     */
    void display() {
        // Print the roll number stored in this object.
        System.out.println("Roll Number: " + rollNumber);
        // Print the name stored in this object.
        System.out.println("Student Name: " + studentName);
        // Print the shared static college name (same for every student).
        System.out.println("College Name: " + collegeName);
        // Print a blank line to separate each student's details.
        System.out.println();
    }

    /**
     * Static method to display the total number of students created.
     */
    static void totalStudents() {
        // Print the shared static counter that counts all Student objects.
        System.out.println("Total Students: " + count);
    }
}

/**
 * Demonstrates the use of the static keyword in a Student Management System.
 */
public class StaticKeyword {

    /**
     * Main method: creates three Student objects to demonstrate static members,
     * then displays each student and the total student count.
     */
    public static void main(String[] args) {
        // Create the first Student using the parameterized constructor.
    }
}
```

```
Student s1 = new Student(1, "Harkeerat");
// Create the second Student using the parameterized constructor.
Student s2 = new Student(2, "Rahul");
// Create the third Student using the parameterized constructor.
Student s3 = new Student(3, "Priya");

// Display the details of the first student.
s1.display();
// Display the details of the second student.
s2.display();
// Display the details of the third student.
s3.display();

// Static method can be called directly using the class name
// Call the static method using the class name (no object needed).
Student.totalStudents();
}
}
```

StudentRecord.java

```
// Import Scanner: used to read student details (ID, name, course, marks) typed on the keyboard.
import java.util.Scanner;
```

```
/**
 * Student class representing a student record with ID, name, course
 * and marks in three subjects. Demonstrates default and parameterized constructors.
 */
class Student {
    // Unique identifier assigned to the student.
    int studentID;
    // Name of the student.
    String studentName;
    // The course in which the student is enrolled.
    String course;
    // Marks secured by the student in the AAS subject.
    double marksAAS;
    // Marks secured by the student in the JAVA subject.
    double marksJAVA;
    // Marks secured by the student in the Python subject.
    double marksPython;

    /**
     * Default constructor: creates an empty Student record using placeholder values.
     */
    Student() {
        // Store 0 as a placeholder ID because none was supplied.
        studentID = 0;
        // Store "Unknown" as the name because no name was supplied.
        studentName = "Unknown";
        // Store "Not Assigned" as the course because no course was supplied.
        course = "Not Assigned";
        // Start all three subject marks at 0.
        marksAAS = 0;
        marksJAVA = 0;
        marksPython = 0;
    }

    /**
     * Parameterized constructor: creates a Student using all the values passed in.
     */
    Student(int id, String name, String c, double aas, double java, double python) {
        // Copy the supplied ID into the instance field.
        studentID = id;
        // Copy the supplied name into the instance field.
        studentName = name;
        // Copy the supplied course into the instance field.
        course = c;
        // Store the AAS marks in the instance field.
        marksAAS = aas;
        // Store the JAVA marks in the instance field.
        marksJAVA = java;
        // Store the Python marks in the instance field.
        marksPython = python;
    }

    /**
     * Accepts the student details from the user through the keyboard.
     */
    void acceptDetails() {
```

```

// Create a Scanner object that reads from System.in (the keyboard).
Scanner sc = new Scanner(System.in);
// Ask the user to enter the student ID.
System.out.print("Enter Student ID: ");
// Read the integer (student ID) typed by the user and store it in the field.
studentID = sc.nextInt();
// Consume the leftover newline character so the next read works correctly.
sc.nextLine();
// Ask the user to enter the student name.
System.out.print("Enter Student Name: ");
// Read the whole line (the name) and store it in the field.
studentName = sc.nextLine();
// Ask the user to enter the course.
System.out.print("Enter Course: ");
// Read the whole line (the course) and store it in the field.
course = sc.nextLine();
// Ask the user to enter the marks in AAS.
System.out.print("Enter Marks in AAS: ");
// Read the double (AAS marks) and store it in the field.
marksAAS = sc.nextDouble();
// Ask the user to enter the marks in JAVA.
System.out.print("Enter Marks in JAVA: ");
// Read the double (JAVA marks) and store it in the field.
marksJAVA = sc.nextDouble();
// Ask the user to enter the marks in Python.
System.out.print("Enter Marks in Python: ");
// Read the double (Python marks) and store it in the field.
marksPython = sc.nextDouble();
}

/**
 * Calculates and returns the total marks in all three subjects.
 */
double calculateTotal() {
    // Sum the marks of AAS, JAVA and Python and return the result.
    return marksAAS + marksJAVA + marksPython;
}

/**
 * Calculates and returns the percentage (total marks divided by the number of subjects).
 */
double calculatePercentage() {
    // Divide the total marks by 3 (three subjects) to get the percentage.
    return (calculateTotal() / 3);
}

/**
 * Displays all the student details along with the total and percentage.
 */
void displayDetails() {
    // Print the student ID stored in the object.
    System.out.println("Student ID: " + studentID);
    // Print the student name stored in the object.
    System.out.println("Student Name: " + studentName);
    // Print the course stored in the object.
    System.out.println("Course: " + course);
    // Print the marks in AAS.
    System.out.println("Marks in AAS: " + marksAAS);
    // Print the marks in JAVA.
    System.out.println("Marks in JAVA: " + marksJAVA);
    // Print the marks in Python.
    System.out.println("Marks in Python: " + marksPython);
    // Print the total marks calculated by calculateTotal().

```

```

        System.out.println("Total Marks: " + calculateTotal());
        // Print the percentage calculated by calculatePercentage().
        System.out.println("Percentage: " + calculatePercentage() + "%");
    }
}

/**
 * Demonstrates classes and objects using default and parameterized constructors.
 */
public class StudentRecord {

    /**
     * Main method: creates two Student objects (one via each constructor),
     * accepts details and displays the records.
     */
    public static void main(String[] args) {
        // Create the first student using the default constructor (all placeholder values).
        Student s1 = new Student();
        // Fill the first student's details through the keyboard.
        s1.acceptDetails();
        // Print a header showing that this is the default-constructor student.
        System.out.println("\n--- Student 1 (Default Constructor) ---");
        // Display the complete record of the first student.
        s1.displayDetails();

        // Create the second student using the parameterized constructor with fixed values.
        Student s2 = new Student(101, "Harkeerat", "BSC-CS", 85.5, 90.0, 78.5);
        // Print a header showing that this is the parameterized-constructor student.
        System.out.println("\n--- Student 2 (Parameterized Constructor) ---");
        // Display the complete record of the second student.
        s2.displayDetails();
    }
}

```